# BÀI KIỂM TRA CHUYÊN MÔN

## P. Phát triển Sản phẩm số - K. Digital

*Vị trí: Backend Developer*

### Hướng dẫn và lưu ý khi làm bài:

- **Thời gian làm bài:** từ 2 đến 3 ngày. Anh/Chị có thể chủ động ước tính thời gian hoàn thành và thông báo lại cho đầu mối tuyển dụng.
- **Thời hạn nộp bài:** chậm nhất 48 giờ trước thời điểm phỏng vấn.
- **Hình thức nộp bài:** Anh/Chị chủ động lựa chọn hình thức phù hợp (ví dụ: đường dẫn GitHub, sản phẩm hoàn chỉnh, …) và gửi kèm hướng dẫn cài đặt/chạy thử (nếu có). Cách trình bày và hình thức nộp bài chỉn chu sẽ được ghi nhận là điểm cộng.
- **Mục đích bài kiểm tra:** là cơ sở để Hội đồng phỏng vấn trao đổi sâu hơn cùng Anh/Chị về cách tiếp cận và giải pháp trong buổi phỏng vấn. Kết quả bài làm không ảnh hưởng đến quyết định mời Anh/Chị tham gia phỏng vấn.
- **Hỗ trợ:** Trong quá trình làm bài, nếu cần làm rõ yêu cầu của đề bài, Anh/Chị vui lòng liên hệ đầu mối tuyển dụng để được kết nối với nhóm giải đáp.

*Cảm ơn Anh/Chị đã dành thời gian thực hiện bài kiểm tra. Chúc Anh/Chị hoàn thành tốt!*

---------- o0o ----------

## Chủ đề: Thiết kế cơ chế đồng bộ Watchlist đa thiết bị

**BỐI CẢNH**

Nền tảng giao dịch của chúng tôi có tính năng Watchlist: danh sách các mã user quan tâm. User có thể thêm mã, xóa mã, sắp xếp lại thứ tự, và gắn ghi chú cho từng mã. Watchlist đồng bộ giữa Web và Mobile.

Ba tuần trước, một khách VIP gặp chuyện: anh ấy mở app trên tàu điện (mất mạng), thêm 3 mã mới. App lưu tạm trên máy, chờ có mạng sẽ đồng bộ. Về đến nhà, anh mở Web: xóa 2 mã cũ đã chốt lời, kéo mã tâm đắc nhất lên đầu danh sách, sửa lại vài ghi chú. Mọi thứ đẹp đẽ.

22 phút sau, điện thoại trong túi bắt được Wi-Fi và lặng lẽ đồng bộ. Toàn bộ thao tác trên Web bay màu: 2 mã đã xóa sống lại, thứ tự trở về như cũ, ghi chú mới biến mất. Khách gọi tổng đài, và câu anh ấy nói được ghi lại nguyên văn trong ticket: *"App của các em phản bội trí nhớ của anh."*

Nguyên nhân gốc: Cơ chế đồng bộ hiện tại theo kiểu bản nào ghi sau thì thắng (last-write-wins) trên toàn bộ danh sách. Bạn được giao thiết kế lại cơ chế đồng bộ này để giúp khách hàng giải quyết âu lo bên trên của họ.

**Câu hỏi 1**

Hãy định nghĩa các API cần thiết cho cơ chế đồng bộ watchlist giữa nhiều thiết bị của một user.

- Không cần implement, chỉ cần định nghĩa rõ (URL, HTTP Method, Request Payload, Response Payload).
- Client làm sao biết bản của mình đã cũ so với server? Thiết kế cơ chế đó vào API.
- Không cần định nghĩa các mã lỗi HTTP tiêu chuẩn (4xx, 5xx).

**Câu hỏi 2**

Hệ thống cần thiết kế 1 class để có thể trả ra danh sách Watchlist sau khi tiến hành merge. Hãy implement Core Logic này bằng Java, trước hết cho các trường hợp không xung đột. *(Yêu cầu thực hiện code apply theo TDD và Clean Architecture.)*

- Các loại thay đổi cần hỗ trợ: `ADD(mã)`, `REMOVE(mã)`, `UPDATE_NOTE(mã, ghi chú)`, `MOVE(mã, vị trí)`.
- Case bắt buộc phải có test: cả A và B cùng `ADD` một mã giống nhau — kết quả là 1 mã hay 2 mã?
- Case đáng suy nghĩ: `MOVE` của bạn lưu "vị trí" theo kiểu gì — index số (vị trí 3) hay tương đối (đứng sau mã FPT)? Nếu A xóa một mã làm mọi index phía sau lệch đi 1, lựa chọn của bạn còn đúng không?

**Câu hỏi 3**

Bây giờ đến xung đột thật sự. Hai tình huống:

- Tình huống A: Thiết bị A xóa mã VNM. Cùng lúc đó (trước khi kịp sync), thiết bị B sửa ghi chú của chính mã VNM thành *"chờ về 80 thì múc"*. Merge xong, kết quả nên là gì — VNM biến mất cùng ghi chú tâm huyết, hay VNM ở lại? Ai thắng, và vì sao lựa chọn đó tốt hơn cho user (không phải tốt hơn cho code)?
- Tình huống B: Giả sử để đơn giản, bạn không lưu thao tác — chỉ so sánh *danh sách cuối cùng* của hai thiết bị. Mã VNM có trong bản của B nhưng không có trong bản của A. Câu hỏi: làm sao bạn biết đây là "A đã xóa VNM" hay "B vừa thêm VNM"? Hai cách hiểu này đòi hỏi kết quả merge ngược nhau hoàn toàn. Tình huống này tiết lộ giới hạn gì của cách tiếp cận "chỉ nhìn trạng thái cuối"?

**Câu hỏi 4**

Dưới đây là log trích xuất từ đúng sự cố của vị khách VIP trong phần bối cảnh. Hãy đọc và trả lời — **không cần code, cần lập luận**:

**Log local — thiết bị Mobile**

```
10:02:11  network=OFFLINE
10:03:40  local op: ADD "HPG"      (queued)
10:04:02  local op: ADD "SSI"      (queued)
10:05:17  local op: ADD "VCI"      (queued)
10:41:00  network=ONLINE
10:41:02  SYNC → PUT /watchlist  baseVersion=17
          body: fullList=[VNM, FPT, MWG, HPG, SSI, VCI]
10:41:02  SYNC ← 200 OK  newVersion=21
```

🖥 Log server

```
10:20:45  PUT /watchlist u=VIP007 dev=web    base=17 op=REMOVE "VNM"          → v18
10:22:03  PUT /watchlist u=VIP007 dev=web    base=18 op=REMOVE "FPT"          → v19
10:23:11  PUT /watchlist u=VIP007 dev=web    base=19 op=MOVE "MWG" → pos 0    → v20
10:41:02  PUT /watchlist u=VIP007 dev=mobile base=17 fullList=[VNM,FPT,MWG,HPG,SSI,VCI]
          ⚠ baseVersion mismatch (client=17, server=20) — accepted anyway (LWW)  → v21
```

Với các solution mà bạn đã thiết kế ở câu hỏi 1, 2, 3 thì theo bạn lúc đấy log hệ thống sẽ thay đổi thế nào khi gặp case tương tự.

### Câu hỏi 5

Bạn trình bày giải pháp đồng bộ mới lên team. Blossom (Đào) — Product Owner — nghe xong bạn estimate 2 tuần thì nhắn lại:

> *"Em ơi, sao phức tạp vậy? Bản nào lưu **sau cùng** thì lấy bản đó thôi — dữ liệu mới nhất là dữ liệu đúng nhất chứ nhỉ? Chị thấy làm vậy 2 ngày là xong mà 😅"*

Hãy viết câu trả lời cho Blossom, với các ràng buộc:

- **Không dùng bất kỳ thuật ngữ kỹ thuật nào** — không "version", không "merge", không "conflict", không "sync". Blossom phải hiểu được và đồng ý được.
- Kết thúc bằng một đề xuất mang tính xây dựng (không chỉ nói "không được đâu chị").